

Desenvolvimento *Web* III

Tecnologia em Análise e Desenvolvimento de Sistemas

Aula 9 – Acesso a banco de dados (3)

Introdução

Veremos hoje mais alguns recursos com o Prisma.

No exemplo da aula anterior, fizemos as operações CRUD básicas para um livro e uma editora.

Recupere o projeto que está disponível no SUAP, e siga os seguintes passos:

1. Execute o comando **npm install** para fazer o *download* do código dos módulos utilizados;

Introdução

2. Inicie o Docker;
3. Suba um *container* do Docker com a imagem do Postgre utilizada:

```
docker run --name postgres -e POSTGRES_PASSWORD=senhadobanco -p 5432:5432 -v postgres:/var/lib/postgresql/data -d postgres:17
```

Introdução

4. Execute o comando **npx prisma db push** para gerar o código de acesso ao banco de dados e enviar as alterações (de esquema) para a base de dados;
5. Execute o comando **npx prisma generate** para gerar os arquivos de acesso;
6. Execute o servidor com o comando **npm run dev**.

O *site* deve funcionar como na aula anterior.

Introdução

Para os exemplos desta aula, crie um novo botão em “livros/page.js”:

```
<button className="ml-2 bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700 active:bg-blue-800 disabled:bg-blue-200" disabled={busy} onClick={handleAcao}>  
  {busy ? " ..... " : "Outro"}  
</button>
```

Crie uma função para ser executada no seu clique:

```
const handleAcao = async () => {  
  
}
```

Introdução

Em “livros/actions.js”, crie uma função para ser executar códigos diversos:

```
export async function Acao(args) {  
  let resultado = null;  
  console.log(resultado);  
}
```

Introdução

E faça sua chamada em page.js na função criada anteriormente:

```
const handleAcao = async () => {  
  await Acao();  
}
```

Não esqueça de importar a função:

```
import { Acao, Pesquisar, Remover } from "../actions";
```

Buscar apenas a primeira tupla

O comando **findFirst** do Prisma recupera apenas a primeira tupla que satisfaz a condição informada.

```
export async function Acao(args) {  
  let resultado = await prisma.livro.findFirst({  
    where: {  
      id: {  
        gt: 10  
      }  
    }  
  });  
  console.log(resultado);  
}
```


Retornando quantidades

O comando **count** do Prisma retorna a quantidade de tuplas que satisfazem a condição informada.

```
export async function Acao(args) {  
  let resultado = await prisma.livro.count({  
    where: {  
      editoralId: {  
        not: null  
      }  
    }  
  })  
  console.log(resultado);  
}
```

Ignorando caixa alta/baixa em pesquisa

Para ignorar caixa alta/baixa na pesquisa de campos de texto, utilize a opção *insensitive* do atributo *mode*:

```
export async function Acao(args) {  
  let resultado = await prisma.livro.count({  
    where: {  
      titulo: {  
        equals: 'estruturas de dados',  
        mode: "insensitive"  
      }  
    }  
  })  
  console.log(resultado);  
}
```

Inserindo vários itens

Para inserir vários itens com um único comando, utilize o comando **createMany** do Prisma:

```
export async function Acao(args) {  
  const livros = [  
    {  
      titulo: 'livro 1',  
      anoPublicacao: 2021,  
      editorald: null  
    },  
    {  
      titulo: 'livro 2',  
      anoPublicacao: 2022,  
      editorald: null  
    }  
  ];  
  
  const resultado = await prisma.livro.createMany({  
    data: livros  
  });  
  
  console.log(resultado.count);  
}
```

Removendo vários itens

O comando **delete** utilizando anteriormente remove pela chave primária:

```
export async function Acao(args) {  
  const resultado = await prisma.livro.delete({  
    where: {  
      id: 5  
    }  
  });  
  
  console.log(resultado);  
}
```

Removendo vários itens

Para remover por campos que não sejam únicos, utilize o comando **deleteMany**.

```
export async function Acao(args) {  
  const resultado = await prisma.livro.deleteMany({  
    where: {  
      editoralId: 3  
    }  
  });  
  
  console.log(resultado);  
}
```

Atualização ou criação de item

Para atualizar um item existente (ou criar caso ele não exista), use o comando **upsert**:

```
export async function Acao(args) {  
  const resultado = await prisma.livro.upsert({  
    where: {  
      id: 1  
    },  
    create: {  
      titulo: 'Estruturas de dados',  
      anoPublicacao: 2020  
    },  
    update: {  
      anoPublicacao: 2020  
    }  
  })  
  console.log(resultado);  
}
```

Buscando em conjunto

Em relacionamentos do tipo 1:N, podemos buscar itens dentro do conjunto relacionado de acordo com três regras:

- **none**, que indica que nenhum item do conjunto deve corresponder ao filtro informado;
- **some**, que indica que ao menos um item do conjunto deve corresponder ao filtro informado;
- **every**, que indica que todos os itens do conjunto deve corresponder ao filtro informado.

Buscando em conjunto

Exemplo: retornar todas as editoras que não possuem livro publicado em 2010.

```
export async function Acao(args) {  
  const resultado = await prisma.editora.findMany({  
    where: {  
      livros: {  
        none: {  
          anoPublicacao: 2010  
        }  
      }  
    }  
  })  
  console.log(resultado);  
}
```


Buscando em conjunto

Exemplo: retornar todas as editoras que possuem ao menos um livro publicado em 2010.

```
export async function Acao(args) {  
  const resultado = await prisma.editora.findMany({  
    where: {  
      livros: {  
        some: {  
          anoPublicacao: 2010  
        }  
      }  
    }  
  })  
  console.log(resultado);  
}
```

Buscando em conjunto

Exemplo: retornar todas as editoras que possuem todos os livros publicados em 2010.

```
export async function Acao(args) {  
  const resultado = await prisma.editora.findMany({  
    where: {  
      livros: {  
        every: {  
          anoPublicacao: 2010  
        }  
      }  
    }  
  })  
  console.log(resultado);  
}
```

Buscando em conjunto

O filtro de pesquisas com o campo `where` não se aplicam a `select`. Por exemplo, se pesquisarmos todas as editoras que possuem ao menos um livro publicado em 2012, e usarmos o `select` para trazer os livros, todos serão recuperados, inclusive os que não forem publicados em 2012.

Para listar apenas os de 2012, um novo filtro deve ser feito dentro do `select`.

Buscando em conjunto

```
export async function Acao(args) {  
  const resultado = await prisma.editora.findMany({  
    where: {  
      livros: {  
        some: {  
          anoPublicacao: 2012  
        }  
      }  
    },  
    select: {  
      nome: true,  
      livros: {  
        select: {  
          titulo: true,  
          anoPublicacao: true  
        }  
      }  
    }  
  })  
  console.log(resultado);  
}
```

Retorna o nome de todas as editoras que tiveram ao menos um livro publicado em 2012, juntamente com todos os seus livros, independente do ano em que foram publicados.

Buscando em conjunto

```
export async function Acao(args) {  
  const resultado = await prisma.editora.findMany({  
    where: {  
      livros: {  
        some: {  
          anoPublicacao: 2012  
        }  
      }  
    },  
    select: {  
      nome: true,  
      livros: {  
        select: {  
          titulo: true,  
          anoPublicacao: true  
        },  
        where: {  
          anoPublicacao: 2012  
        }  
      }  
    }  
  })  
  console.log(resultado);  
}
```

Retorna o nome de todas as editoras que tiveram ao menos um livro publicado em 2012, juntamente com todos os seus livros que foram publicados em 2012.

Transações

Para fazer várias consultas em uma mesma transação, use o objeto **\$transaction** do Prisma.

Exemplo: retornar a quantidade total de livros, a quantidade de livros sem editora e a quantidade de livros com editora. Realizar na mesma transação para garantir que outras transações não executem entre as consultas desta transação.

Transações

```
export async function Acao(args) {  
  const [livrosTotal, livrosSemEditora, livrosComEditora] = await prisma.$transaction(  
    [  
      prisma.livro.count(),  
      prisma.livro.count({  
        where: {  
          editorald: null  
        }  
      }),  
      prisma.livro.count({  
        where: {  
          NOT: {  
            editorald: null  
          }  
        }  
      })  
    ]  
  );  
  
  console.log({  
    total: livrosTotal,  
    semEditora: livrosSemEditora,  
    comEditora: livrosComEditora  
  });  
}
```

Migrações

Até agora utilizamos o comando **prisma db push** para sincronizar o banco de dados com as *models* criadas no arquivo `schema.prisma`.

Esse processo é rápido, mas no entanto não é rastreável nem verificável, normalmente sendo utilizado em desenvolvimento mas não em produção.

Migrações

Ao alterar o *schema* do banco de dados normalmente criamos uma migração, a qual podemos ver o código que foi utilizado para fazer alterações se necessário.

O Prisma não possui, atualmente, um comando de *rollback* para realizar o *rollback* de uma migração.

Para criar uma migração, utilizamos o comando:

```
npx prisma migrate dev --name nomeMigracao
```

Migrações

O comando anterior gerará um código sql na pasta “migrations”, que você pode conferir antes de executar.

Crie uma migração inicial. Pode ser necessário reiniciar a base de dados:

```
npx prisma migrate dev --name init
```

Migrações

Adicione o campo Autor na *model* Livro em schema.prisma:

```
model Livro {  
  id          Int @id @default (autoincrement())  
  titulo      String  
  anoPublicacao Int  
  editorald   Int ?  
  editora     Editora ? @relation(fields: [editorald], references: [id])  
  autor       String ?  
}
```

Migrações

Crie uma nova migração com o comando:

`npx prisma migrate dev --name autor`

O banco de dados será atualizado e os arquivos locais também, não sendo necessário utilizar o comando *generate* para os atualizar.

Prisma Studio

É possível utilizar o Prisma Studio para visualizar os dados, realizar consultas SQL no navegador, entre outros, de forma a facilitar o desenvolvimento.

Para isso, utilize o comando:

`npx prisma studio`

Exercício

Criar um *site* para registro de diários escolares, sendo:

- Docente (nome, e-mail) pode ser vinculado a diversas disciplinas;
- Disciplina (código, nome) pode ser de apenas um curso;
- Curso (código, nome, ano de início) pode ter diversas disciplinas.

Exercício

Realizar o CRUD básico de docente;

Realizar o CRID básico de curso;

Permitir associar disciplinas (criar, remover, atualizar, etc.) a um curso;

Listar disciplinas por docente;

Listar cursos que cada docente atua;

Listar todos os docentes de um curso.